Informe de Conclusiones

Del dato al análisis – Variante 3: Estacionalidad y ritmo de la operación.

0.Introducción

En este informe se presentan las decisiones, lo encontrado con los datos y hasta donde se puede afirmar algo en base a ellos. Partimos de la base migrada del Trabajo Práctico 1, sakila_espanol, con su esquema sakila.

1.Pregunta de negocio y como la abordamos

Nuestra variante tiene dos preguntas: como se distribuye la demanda en el tiempo y si hay un patrón semanal para planificar personal y promociones.

Trabajamos con la tabla alquiler y pago. Decidimos desde un inicio armar una serie diaria de alquileres e ingresos usando un calendario completo, comprendiendo los días operacionalmente asentados. Sobre esa serie se realizó el cálculo de la métrica derivada aplicando las técnicas exigidas.

A causa de las observaciones obtenidas en el punto 2, que en un inicio nos llevaron a desestimar solamente los extremos registrados que parecían no ser representativos, se obtuvo como resultado hallazgos que tras una revisión fueron erróneos de afirmar. Como, por ejemplo, que la métrica de los martes presentaba un decaimiento representativo en comparación al domingo el cual superaba la media por 14 puntos.

Ante estos resultados que ofrecían respuestas a nuestras preguntas de negocio e ideas para aprovechar promociones, surgieron dudas. Las cuales nos llevaron a revisar manualmente contra el origen, observando en esta ocasión que el patrón de carga del primer martes de cada bloque se efectuaba en un rango horario (1 hora), el cual no era comparable contra el resto de los rangos (24 horas), lo que nos llevó a considerar estos seis días como (parciales). Por lo tanto, decidimos modificar nuestro cálculo de métrica derivada excluyéndolos del análisis, de esta manera obtuvimos otros resultados que nos permiten realizar comparaciones más realistas sobre las cuales decidimos trabajar y responder a las preguntas de negocio como:

La demanda de alquileres en un bloque semanal se distribuye de una manera pareja. Ahora, la diferencia se ve reflejada entre los bloques operativos los cuales llegan a cuadruplicar la demanda entre el primer bloque en junio y último bloque operacional en agosto.

El viernes, sábado y domingo son los días que llegan y/o sobrepasan el promedio en porcentajes muy pequeños, por lo tanto, no los consideramos como patrones para aprovechar promociones ni aumentar personal, sino más bien aumentar el catálogo de alquileres ya que con la observación del ticket promedio podemos ver que los ingresos son parejos y proporcionales al caudal de alquileres que se produce. Es decir, se podrían aprovechar esos tres días para ofertar una mayor cantidad de películas.

2.Decisiones de diseño

2.1 Reparto del trabajo entre SQL Server y Pandas

Siguiendo la regla de la U5, delegamos al motor lo que puede resolver de forma más eficiente:

- La unión entre alquiler y pago con un LEFT JOIN, para no perder los alquileres sin pago.
- El filtro por rango de fechas, que viaja por parámetros.
- La agregación por día con COUNT(DISTINCT), para que un alquiler con más de un pago no cuente el doble.

Pandas recibe los datos ya filtrados y reducidos desde SQL y se encarga del tipado, el perfilado, el reindexado sobre el calendario, la media móvil, el resumen semanal y el índice.

Al realizar una medición de las mismas métricas por ambos caminos y validarlo con pd.testing.assert_frame_equal(), ratificamos la decisión, ambos enfoques producen exactamente la misma salida, pero el del motor resulta notoriamente más eficiente al devolver los datos previamente filtrados.

2.2 Seguridad y portabilidad

La conexión usa autenticación integrada de Windows y lee el servidor y la base desde un archivo. env que no se sube al repositorio, por lo que no hay ninguna credencial escrita en el código. Todos los valores variables viajan por el parámetro params de pandas.read_sql, sin armar texto concatenado.

Los nombres de tablas y columnas son un caso aparte, porque son identificadores y no valores, el motor SQL no permite pasarlos como parámetros comunes. Lo resolvimos en tres pasos dentro de conjunto_trabajo.sql: cada nombre llega como parámetro desde esquema.py, se valida contra el catálogo del motor(sys.columns) y, si no existe, la consulta se frena con un error propio. En ese momento se inserta con QUOTENAME, y las fechas viajan por sp_executesql. Por lo que, cambiando solo esquema.py, el trabajo puede correr sobre la base de otro equipo.

2.3 Elección del controlador

Elegimos pyodbc con el ODBC Driver 17 for SQL Server, en lugar de mssql-python. Fue el camino que consideramos más seguro y maduro para trabajar con pandas, debido a lo recomendado en la U5, y además nos permitió fast_executemany, que agrupa las inserciones en lotes beneficiándonos cuando cargamos la tabla destino.

2.4 Criterios de limpieza

El diagnostico no encontró faltantes, duplicados ni reglas de negocios violadas, así que no descartamos ninguna fila. Las decisiones fueron:

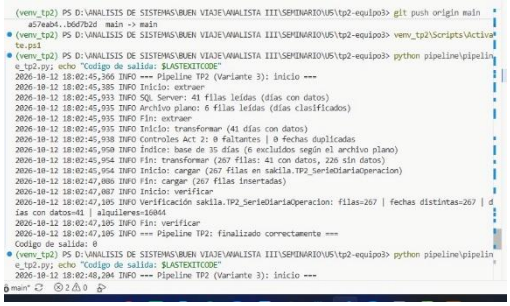
- Tipado: la fecha llegó como texto(object) y la pasamos a tipo fecha.

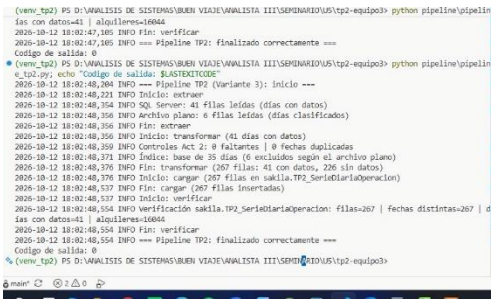
- Huecos: los días sin datos quedan como valor ausente (NaN) y no como 0. Un 0 diría que no hubo alquileres, y lo único que sabemos es que no hay registros.
- Días no representativos: Los 5 martes de apertura de bloque y el 14/02/2006 se mantienen en la serie, pero no entran al índice de estacionalidad.
- Controles: faltantes y duplicados se siguen revisando en cada corrida del pipeline, y si aparece quedan en el log.

2.5 Carga e Idempotencia

El pipeline (pipeline/pipeline_tp2.py) se ejecuta por línea de comandos y está dividido en funciones con una sola tarea: extraer, transformar, cargar y verificar, más ejecutar_pipeline que las coordina. Combina con merge dos orígenes, la base en SQL Server y el archivo plano de equipo, y carga en en sakila_espanol 'TP2_SerieDiariaOperacion' una fila por cada día del calendario: 267 filas, incluidos los días sin datos.

La idempotencia la logramos reemplazando todo en cada corrida: to_sql con if_exists="replace", con los tipos de columna fijados con dtype, y todo dentro de una sola transacción. Si algo falla a la mitad, se deshace y la tabla queda como estaba. Además, verificar() cuenta lo que se cargó y frena el pipeline si la cantidad de filas no coincide o si hay fechas repetidas. Cada etapa se registra con logging, y el proceso completo está en un try/except con log. exception y raise, por lo que si falla devuelve un código de salida distinto de cero. Evidencia de las dos ejecuciones consecutivas. En ambas corridas, la línea de verificación fue idéntica:







3.Calidad de datos

El conjunto de trabajo tiene 41 filas, una por cada día con actividad, que suman 16.044 alquileres y $67.406,56 de ingresos. Estos fueron los controles:

Control	Resultado	Decisión
Faltantes y duplicados por fecha	0 y 0	Sin corrección
Atípicos (rango intercuartílico)	Q1 174, Q3 621, IQR 447; límites [-496,5; 1.291,5]; 0 atípicos	Sin corrección
Reglas de negocio	0 violaciones	Sin corrección
Perfilado en el motor contra pandas	Los 3 controles coinciden (duplicados 0, regla 0, días sin datos 226)	-
Huecos en la serie (control especifico)	267 días de calendario: 41 días con datos (15, 36%) y 226 sin datos (84, 64%)	NaN, nunca 0

En el control especifico de la variante, el periodo que cubren los datos no es continuo ni es un año completo. Los datos vienen en 5 bloques de 8 días operacionales, que van de un martes a la noche (alrededor de las 22:50) hasta el martes siguiente y, empiezan cada 21 días; el 24/05, 14/06, 05/07, 26/07 y 16/08 del 2005. Los bloques están separados por 4 brechas de 13 días sin registros, seguidas de 174 días hasta un último y único día aislado, el 14/02/2006.

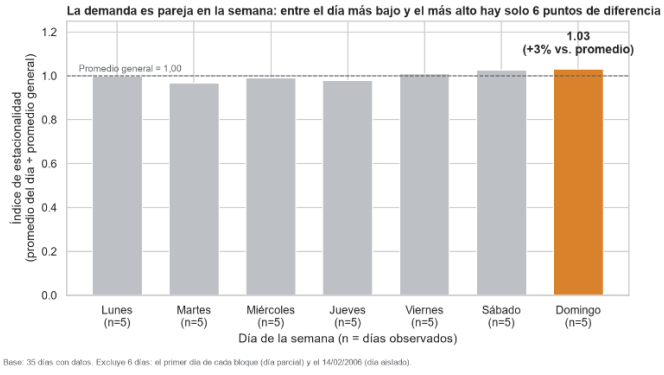
Los martes de apertura de cada bloque solo tienen actividad desde 22:49-22:53, con 8, 16, 27, 33 y 23 alquiles, así que son días parciales y no de poca demanda. El 14/02/2006 tiene 182 alquileres, todos a las 15:16:03 y ninguno devuelto, lo que parece una carga de lote y no una operación real.

Impacto en las conclusiones:

- La inconsistencia en la periodicidad de las fechas asentadas limita cualquier análisis más escalable, porque impide estudiar tendencias anuales, semestrales o mensuales de forma confiable; la serie solo admite evaluaciones semanales.
- Los días parciales distorsionan: si los incluyéramos el martes bajaría de 0,97 a 0,57 y aparecería un "martes flojo" que los datos no sostienen. Por eso los excluimos.
- Los huecos no deforman la media: al tratarlos como ausentes, la MM7 pide al menos 4 días reales en la ventana y no se calcula en los días sin datos.

4.Hallazgos

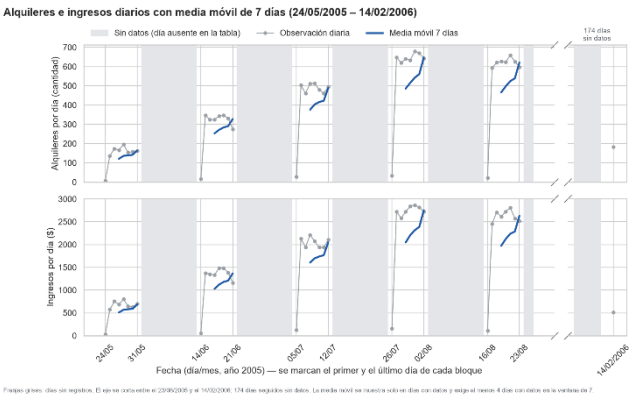
4. 1: La semana es pareja, no hay un día que concentre la demanda

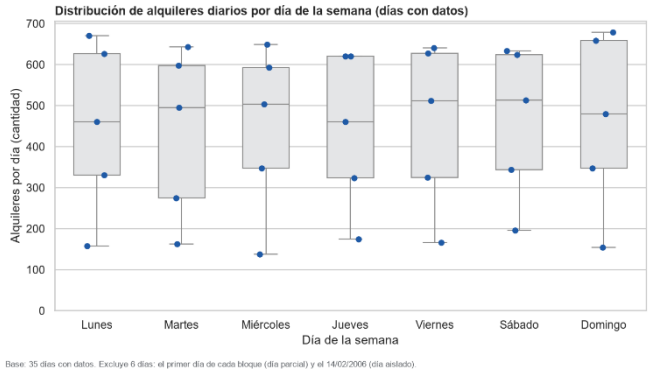


Observación: el índice de alquileres va de 0,966(martes) a 1,031(domingo), ningún día se aleja más de un 3,5% del promedio general.

Explicación: dentro de cada bloque la demanda se reparte casi igual entre los días, y la diferencia entre los extremos es pequeña comparada con el volumen diario. ***Decisión que se habilita:*** personal planificado de forma pareja durante la semana, mantener precios base estables. Como estrategia complementaria: aplicar un incremento de precio leve, el fin de semana (+5 %) para aprovechar el incremento sin afectar la rotación.

4.2: El ritmo cambia entre bloques, no entre días de la semana





Base: 35 días con datos. Excluye 6 días: el primer día de cada bloque (día parcial) y el 14/02/2006 (día aislado).

Observación: La MM7 al cierre de cada bloque pasa de 164 en mayo a 328 en junio, 489 y 648 en julio, y 621 en agosto. La variación entre bloques es mucho mayor que la diferencia entre días de la semana dentro de un mismo bloque. ***Explicación***: el volumen se cuadruplica en poco más de dos meses. Dentro de este marco, la variación relevante para la operación es la del período, no la del día de la semana.

Decisión habilitada: el personal, stock y la oferta disponible deben dimensionarse por periodo, no por día, y usando la MM7 como referencia para planificar capacidad y abastecimiento siguiendo su tendencia.

4.3: El ticket promedio es estable, los ingresos acompañan al volumen

Observación: el ingreso por alquiler, según el día de la semana, se mantiene entre $4.15 y $4.30, y los índices de ingresos siguen a los de alquileres (domingo 1,027; martes 0,971). ***Explicación***: el monto de cada alquiler casi no varía, así que los ingresos de un día dependen sobre todo de cuantos alquileres hubo.

Decisión habilitada: Los ingresos pueden proyectarse directamente a partir de los alquileres esperados. Para aumentar la recaudación, la palanca es el volumen de alquileres.

5. Limitaciones

- ***No se puede hablar de estacionalidad anual***: Los datos cubren 4 meses de 2005 (mayo a agosto) y un día aislado de 2006. Sin al menos 1 año completo no se puede separar el efecto de la época del año de cualquier otra causa. Lo que muestra el hallazgo 2 es un cambio entre bloques, no estacionalidad.
- ***No sabemos que paso en los huecos***: El 84,64% del calendario no tiene datos. No podemos decir que esos días no hubo actividad, solo que no hay registro, así que ningún total representa la operación completa
- ***La muestra semanal es chica***: Cada día de la semana se observa completo 5 veces (35 días en total), y diferencias de alrededor del 3% pueden ser casualidad. El hallazgo 1 dice que no hay un patrón fuerte, pero no que no exista ninguno.
- ***Información incongruente***: La actividad se reparte de forma plana en 24 horas o de golpe en 1h, los bloques regulares de 8 días cada 21 refuerzan una idea

poco realista para un local. Las conclusiones valen para esta base y no para la vida real.

- ***La MM7 no existe en todos los días con datos***: Pedimos 4 días reales en la ventana, aparece recién el cuarto día de cada bloque (asegurando que coincida con la secuencia operacional de 8 días), y solo 25 días del calendario la tienen, ya que incluir todo implicaría comparaciones incongruentes.
- ***La tabla destino no tiene clave primaria***: El reemplazo con to_sql crea las tablas sin restricciones, así que la unicidad de cada fecha la garantiza el control de verificar () y no el motor.

6. Dificultades y resoluciones

6.1. Seguridad de las consultas.

Hicimos tres versiones. La primera armaba parte de las consultas con f-strings. La segunda centralizó los nombres en esquema.py, pero los nombres de tablas y columnas son identificadores y no pueden ir por params. La final los pasa como parámetros a la consulta T-SQL, los valida contra sys.columns y los inserta con QUOTENAME. La única f-string que quedó es la cadena de conexión en config_conexion.py, que no es una consulta ni lleva datos de un usuario.

6.2. Estructura del proyecto.

Python no encontraba config_conexion.py y esquema.py desde la carpeta notebooks/. En el notebook agregamos la carpeta superior a sys.path, y en el pipeline calculamos las rutas desde el propio script. Además, nuestra base usa el esquema sakila y no dbo, así que lo sumamos a esquema.py.

6.3. Día con cero o día ausente.

Al principio rellenamos los huecos con 0, pero eso afirmaba una inactividad que no está registrada y hacía caer la media móvil. Los dejamos como NaN y usamos min_periods=4.

6.4. Qué días excluir del índice.

Primero sacamos solo los dos extremos del período (39 días de base). Al mirar los horarios vimos que el primer martes de cada bloque también es parcial, así que excluimos los 5 martes de apertura más el 14/02/2006 y quedaron 35 días, 5 por día de la semana.

6.5. Agrupar por fecha no es perder información.

Nos preocupaba perder registros al agrupar por día, pero la suma diaria da los 16.044 alquileres del total. El nivel de análisis lo define la pregunta de negocio, y además hace falta una fila por día para usar rolling y resample.

6.6. Formato del archivo plano.

El CSV de SSMS salió con punto y coma y con BOM. En lugar de editarlo a mano, ajustamos la lectura en el pipeline.

6.7. Maquetado del notebook.

Las primeras visualizaciones se veían demasiado compactas, generándonos confusión, al punto de provocarnos la pérdida accidental de código que tuvimos que rehacer. Estandarizamos encabezados y separadores para que se lea más segmentado.

6.8. Trabajo en equipo con distintas herramientas.

En simultaneo, cada uno armaba su versión y después las comparábamos. Tuvimos que elegir y unificar nombres, criterios, las mejores estrategias, y verificar todo contra la base elegida. Llevó más tiempo, pero cada decisión quedó armada por partes de los tres.